Constrained Pseudorandom Functions, Revisited

Julian Mouthon, Mario Razafinony

Master 1 in Cryptology, High-Performance Computing and Algorithms

Sorbonne Université

22 mai 2026

Introduction

Pseudorandom functions (PRFs) are fundamental primitives in modern cryptography. They are used in many protocols, notably for symmetric encryption, authentication, and key derivation. In some contexts, it is necessary to delegate only part of the evaluation capabilities of a PRF without revealing the full secret key. This need arises in particular in the context of Searchable Symmetric Encryption and outsourced storage on untrusted servers, where it is necessary to precisely control an adversary's evaluation capabilities [1]. Constrained pseudorandom functions (CPRFs) address this problem by allowing the restriction of the function's evaluation to a subset of inputs defined by a constraint [1].

This project focuses on the CPRF construction proposed by Bost, Minaud, and Ohrimenko in 2017 [1], based on iteration of a trapdoor permutation. First, we present the fundamental notions related to pseudorandom functions and CPRFs, as well as the construction introduced in [1]. Second, we study several variants of this construction based on RSA, Rabin, and AES, as well as an alternative based on linear transformations, F^{WEAK}, for which we propose a hashed version and analyze its security. Finally, we revisit recent work by Cheng and Jaeger [2], which presents an attack on this CPRF and proposes a hashed variant that improves its security.

The project is accompanied by a complete implementation and simulations allowing experimental evaluation of the studied attacks.

The source code of the project is available on GitHub : https://github.com/julian-mtn/bmo17-cprf.git

Table des matières

1 Definitions

Pseudorandom function (PRF)

A pseudorandom function

$$F : \mathcal{K} \times D \to R$$

is a function such that, for a secret key K, the function $F(K, \cdot)$ is indistinguishable from a truly random function $G : D \to R$ for any efficient adversary.

Constraint

A constraint is a Boolean predicate

$$C : D \to \{0, 1\},$$

which partitions the domain D into two sets :

- *authorized* inputs such that $C(x) = 1$
- *forbidden* (or constrained) inputs such that $C(x) = 0$

Intuitively, the constraint specifies the set of inputs on which a constrained key is allowed to evaluate the pseudorandom function.

Here, we mainly consider constraints of the form

$$C_n(x) = [x < n],$$

which allow all inputs strictly smaller than a threshold $n \in \mathbb{N}$.

Constrained pseudorandom function (CPRF)

A constrained pseudorandom function (CPRF) is defined by four algorithms :

- $\mathsf{Setup}(1^\lambda) \to K$ (master key) : generates the main secret key K.
- $\mathsf{Constrain}(K, C) \to K_C$ (constrained key) : produces a special key K_C that allows evaluation of the function only on inputs authorized by C.
- $\mathsf{Eval}(K, x) \to y$: evaluates the pseudorandom function on input x using the master key.
- $\mathsf{EvalC}(K_C, x) \to y$: evaluates the function on input x using the constrained key K_C.

A CPRF is correct if, for any key K, any constraint C, and any input x such that $C(x) = 1$, we have

$$\mathsf{EvalC}(K_C, x) = \mathsf{Eval}(K, x).$$

In other words, the constrained key allows exact evaluation of the PRF on authorized inputs.

2 Construction of the BMO17 CPRF

2.1 Properties

The BMO17 CPRF is based on the successive iteration of a trapdoor permutation, i.e., a bijective function that is easy to compute but hard to invert without secret information. We denote this permutation by π.

Master key generation

The master key consists of the following elements :

- $ST_0 \in \mathbb{Z}_N$, a secret initial state chosen uniformly at random
- SK, an RSA secret key defining the trapdoor permutation π_{SK}

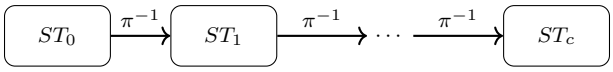
The associated public key PK only allows computation of the forward permutation π.

Evaluation of the CPRF with the master key

With the master key (ST_0, SK) and an input c, the CPRF is evaluated as :

$$Eval((SK, ST_0), c) = \pi_{SK}^{-c}(ST_0)$$

The evaluation consists in applying the inverse permutation c times starting from the initial state ST_0.



Constrained key generation

Starting from the master key (ST_0, SK) and an integer n, corresponding to the constraint $C(c) = [c < n]$, the constrained key is composed of the following elements :

- PK, the public key associated with the permutation π
- $ST_n = \pi_{SK}^{-n}(ST_0)$
- n

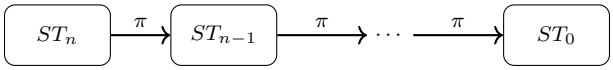
The secret key SK is not included in the constrained key.

Evaluation of the CPRF with the constrained key

With the constrained key (PK, ST_n, n) and an input c, evaluation is possible only if $c < n$. In this case, the CPRF value is computed as :

$$\mathsf{EvalC}((PK, ST_n, n), c) = \pi_{PK}^{n-c}(ST_n)$$

This value is equal to $Eval((SK, ST_0), c)$ by construction, ensuring correctness of the scheme.



2.2 CPRF with RSA trapdoor permutation

We use the OpenSSL library for handling large integers (`BIGNUM`), modular arithmetic, and SHA-256 hashing.

Randomness is securely obtained from `/dev/urandom`, which allows generating the initial state ST_0 as well as RSA parameters in a secure way.

2.2.1 Implementation of the RSA trapdoor permutation

The trapdoor permutation used in BMO17 is defined using RSA. We implement RSA key generation with 4096-bit keys, as well as evaluation of the permutation in both directions.

More precisely, the public function is :

$$x \mapsto x^e \bmod N,$$

while the inverse permutation is computed as :

$$x \mapsto x^d \bmod N.$$

with

$$x^{e \cdot d} = x \bmod N, \forall x \in \mathbb{Z}_N.$$

2.2.2 Implementation of the master key and generation of the initial state

The CPRF master key is composed of a secret initial state ST_0 and an RSA private key. The initial state is generated as a 256-bit random integer using `/dev/urandom`.

Evaluation with the master key consists of applying c iterations of the inverse RSA permutation starting from ST_0.

2.2.3 Implementation of constrained key derivation

Starting from the master key and a constraint parameter n, we derive a constrained key composed of the RSA public key (e, N) and the state

$$ST_n = \pi^{-n}(ST_0).$$

This key then allows evaluating the CPRF only for inputs $c < n$ by applying $(n - c)$ iterations of the public permutation starting from ST_n. The private key is never included in the constrained key, which ensures that only authorized evaluations are possible.

3 Study of other alternative permutations

3.1 Linear permutation $F^{\mathbf{WEAK}}$

In this section, we present an implementation of a CPRF whose input and output are vectors $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$ and $\vec{y} \in (\mathbb{Z}/p\mathbb{Z})^M$, where p is a prime number and N and M are the vector dimensions.

3.1.1 Permutation implementation

Here, the trapdoor permutation evaluation is defined as :

$$Eval(F^{\text{WEAK}}, \vec{x}) = Eval(S, \vec{x}) = S \cdot \vec{x}$$

where S is a matrix of size $M \times N$ with coefficients in $\mathbb{Z}/p\mathbb{Z}$ and $M \ll N$.

S is the master key and is chosen uniformly at random during key generation.

3.1.2 Constrained key implementation

To generate the constrained key :

- the attacker sends a vector $\vec{y} \in (\mathbb{Z}/p\mathbb{Z})^N$
- the oracle samples a random vector $\vec{d} \in (\mathbb{Z}/p\mathbb{Z})^M$ such that $\vec{d} \neq \vec{0}$
- the oracle returns $Constrain(S, \vec{y}) = S_{\vec{y}} = S + d \cdot \vec{y}^T$

By defining

$$CEval(S_{\vec{y}}, \vec{x}) = S_{\vec{y}} \cdot \vec{x}$$

we observe that the set of constrained vectors is exactly the set of $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$ such that $\langle \vec{x}, \vec{y} \rangle = 0$.

Indeed,

$$\begin{aligned} CEval(S_{\vec{y}}, \vec{x}) &= S_{\vec{y}} \cdot \vec{x} \\ CEval(S_{\vec{y}}, \vec{x}) &= S \cdot \vec{x} + d \cdot \vec{y}^T \cdot \vec{x} \\ CEval(S_{\vec{y}}, \vec{x}) &= S \cdot \vec{x} + d \cdot \langle \vec{x}, \vec{y} \rangle \\ CEval(S_{\vec{y}}, \vec{x}) &= S \cdot \vec{x} \iff \langle \vec{x}, \vec{y} \rangle = 0 \end{aligned}$$

Therefore :

$$S_{\vec{y}} \cdot \vec{x} = \begin{cases} S \cdot \vec{x} & \text{if } \langle \vec{x}, \vec{y} \rangle = 0 \\ \text{“noisy” output} & \text{otherwise} \end{cases}$$

This construction restricts evaluation of the function to vectors orthogonal to $\vec{y}$. When the attacker sends a vector $\vec{y}$, the oracle must reject if $\vec{y} = \vec{0}$; otherwise it returns $S_{\vec{y}} = S$, which would reveal the master key to the attacker.

3.1.3 Linear-system based attack approach

Let $\vec{S_i} \in (\mathbb{Z}/p\mathbb{Z})^N$ denote the i-th row of S.

For a vector $\vec{x}$ orthogonal to $\vec{y}$, for each $\vec{S_i}$ we define $k_i = \vec{S_i} \cdot \vec{x} = S_{\vec{y}} \cdot \vec{x}$.

$$\begin{matrix} S & & \vec{x} & & \vec{k} \end{matrix}$$
$$\begin{bmatrix} \\ \vec{S_1} \\ \\ \vec{S_2} \\ \\ \vdots \\ \\ \vec{S_M} \\ \\ \end{bmatrix} \cdot \begin{bmatrix} \\ x_1 \\ \\ x_2 \\ \\ \vdots \\ \\ x_N \\ \\ \end{bmatrix} = \begin{bmatrix} \\ k_1 \\ \\ k_2 \\ \\ \vdots \\ \\ k_M \\ \\ \end{bmatrix}$$

FIGURE 1 – Representation of the linear system $S \cdot \vec{x} = \vec{k}$

By choosing N vectors $\{\vec{x_j}\}_{1 \leq j \leq N}$ such that $\langle \vec{x_j}, \vec{y} \rangle = 0$ for all j, we can construct a matrix $X \in (\mathbb{Z}/p\mathbb{Z})^{N \times N}$ such that the j-th row of X is $\vec{x_j}^T$.

$$\begin{bmatrix} \\ \vec{x_1} \\ \\ \vec{x_2} \\ \\ \vdots \\ \\ \vec{x_N} \\ \\ \end{bmatrix}$$

FIGURE 2 – Matrix X for the attack built from $\{\vec{x_j}\}$ orthogonal to $\vec{y}$

To recover the matrix S, it is therefore sufficient to solve the linear system $X \cdot \vec{Si}^T = \vec{k_i}$ where the j-th element of $\vec{k_i}$ is $k_{i_j} = \vec{Si} \cdot \vec{x_j}$, for each row S_i of S.

$$\begin{matrix} X & & \vec{S_i}^T & & \vec{k_i} \end{matrix}$$
$$\underbrace{\begin{bmatrix} \\ \vec{x_1} \\ \\ \vec{x_2} \\ \\ \vdots \\ \\ \vec{x_N} \\ \\ \end{bmatrix}}_{N \times N} \cdot \underbrace{\begin{bmatrix} \\ S_{i,1} \\ \\ S_{i,2} \\ \\ \vdots \\ \\ S_{i,N} \\ \\ \end{bmatrix}}_{N \times 1} = \underbrace{\begin{bmatrix} \\ k_{i_1} \\ \\ k_{i_2} \\ \\ \vdots \\ \\ k_{i_N} \\ \\ \end{bmatrix}}_{N \times 1}$$

FIGURE 3 – Representation of the linear system $X \cdot \vec{S_i}^T = \vec{k_i}$, where $k_{i_j} = \vec{S_i} \cdot \vec{x_j}$

This procedure is repeated M times in order to recover each $\vec{Si}$.

Problem : The linear system admits a unique solution if and only if the matrix X is invertible, which is equivalent to $\text{rang}(X) = N$. In order to guarantee this invertibility, it is necessary to choose vectors $\vec{x_j}$ such that the family $\{\vec{x_j}\}_{1 \leq j \leq N}$ is linearly independent, that is, it spans a vector space of dimension N.

Furthermore, each vector $\vec{x_j}$ must be orthogonal to $\vec{y}$. We then have that the set $\{\vec{y}\} \cup \{\vec{x_j}\}_{1 \leq j \leq N}$ consists of $N+1$ pairwise linearly independent vectors, and would therefore span a vector space of dimension $N+1$.

This is impossible, since all these vectors belong to $(\mathbb{Z}/p\mathbb{Z})^N$, which has dimension N. This contradiction shows that it is impossible to construct such a family of vectors $\{\vec{x_j}\}_{1 \leq j \leq N}$ simultaneously satisfying these conditions.

Alternative : In the general case, we still observe that $\vec{y}$ spans a vector space of dimension 1, which implies that it is possible to choose $N-1$ vectors $\{\vec{x_j}\}_{1\leq j\leq N-1}$ such that each vector $\vec{x_j}$ is orthogonal to $\vec{y}$.

Indeed, in the previous linear system, for each row j of X, we have :

$$x_{j,1} * S_{i,1} + \ldots + x_{j,N} * S_{i,N} = k_{i_j} \Leftrightarrow x_{j,1} * S_{i,1} + \ldots + x_{j,N-1} * S_{i,N-1} = k_{i_j} - x_{j,N} * S_{i,N}$$

We can therefore reconstruct another matrix X of size $(N-1)\times(N-1)$ and then define the following linear system :

$$\underbrace{\begin{bmatrix} x_{1,1} & \cdots & x_{1,N-1} \\ x_{2,1} & \cdots & x_{2,N-1} \\ \vdots & \vdots & \vdots \\ x_{N-1,1} & \cdots & x_{N-1,N-1} \end{bmatrix}}_{X} \cdot \underbrace{\begin{bmatrix} S_{i,1} \\ S_{i,2} \\ \vdots \\ S_{i,N-1} \end{bmatrix}}_{\vec{S_i}^T} = \underbrace{\begin{bmatrix} k_{i_1} - x_{1,N} \cdot S_{i,N} \\ k_{i_2} - x_{2,N} \cdot S_{i,N} \\ \vdots \\ k_{i_{N-1}} - x_{N-1,N} \cdot S_{i,N} \end{bmatrix}}_{\vec{k_i}}$$

$(N-1)\times(N-1)$ $\quad$ $(N-1)\times 1$ $\quad$ $(N-1)\times 1$

FIGURE 4 – Representation of the new linear system $X \cdot \vec{S_i}^T = \vec{k_i}$, where $k_{i_j} = \vec{S_i} \cdot \vec{x_j}$

Here, the set of vectors $\{\vec{x_j}\}_{1\leq j\leq N-1}$ forms a linearly independent family, therefore $rang(X) = N-1$, hence X is invertible, and thus this linear system has a unique solution. However, this unique solution is computed as a function of $S_{i,N}$.

By repeating this process M times, it is therefore possible to rewrite each row S_i of S as a function of a single element $S_{i,N}$.

This alternative therefore makes it possible to reconstruct each element $S_{i,j}$ of the matrix S as a function of a single element $S_{i,N}$. This allows the attacker, for each query sent to the oracle, to recover one value of $S_{i,N}$, and to keep the values of $S_{i,N}$ that appear most frequently as the true value of $S_{i,N}$.

Remark : The choice of using an element $S_{i,N}$ to express each row S_i of S is arbitrary. Each row S_i of S could have been rewritten using any element $S_{i,l}$. In this case, in the previous linear system, each row $\vec{x_i}$ of X becomes $(x_{i,1},\ldots,x_{i,l-1},x_{i,l+1},\ldots,x_{i,N}), \forall i \neq l$, the vector $\vec{S_i}^T$ becomes $(S_{i,1},\ldots,S_{i,l-1},S_{i,l+1},\ldots,S_{i,N})$, and each element of $\vec{k_i}$ becomes $k_{i_j} - x_{j,l} \cdot S_{i,l}, \forall i \neq l$

Special case : We observe that when the attacker sends a vector $\vec{y} = (0,\ldots,0,1)$ in order to receive the master key $S_{\vec{y}}$, they notice that $\{(1,0,\ldots,0),(0,1,0,\ldots,0),\ldots,(0,\ldots,0,1,0)\}$ are $N-1$ vectors of $(\mathbb{Z}/p\mathbb{Z})^N$ that are all orthogonal to $\vec{y}$.

To reconstruct the matrix $X \in (\mathbb{Z}/p\mathbb{Z})^{(N-1)\times(N-1)}$, they can then choose a vector $\vec{x_j} = (0,\ldots,0,1,0,\ldots,0) \in (\mathbb{Z}/p\mathbb{Z})^{N-1}$ for row j, where the vector contains a 1 at its j^{th} coordinate.

Thus, in the previous linear system, we have $X = Id_{N-1}$ and $\forall j, x_{j,N} = 0$. We therefore obtain the linear system $X \cdot \vec{S_i}^T = \vec{k_i} \Leftrightarrow Id_{N-1} \cdot \vec{S_i}^T = \vec{k_i}$. This makes it possible to immediately recover the first $N-1$ columns of S. Indeed, in this special case, $\forall j \leq N-1, S_{i,j} = k_{i_j} = \vec{S}_{\vec{y_i}} \cdot \vec{x_j} = S_{\vec{y}_{i,j}}$.

Therefore, after requesting the evaluation of a vector $EVAL(\vec{x}) = \vec{y}$ from the oracle, in order to recover $S_{i,N}$, we have $S_{i,1} \cdot x_1 + \cdots + S_{i,N-1} \cdot x_{N-1} + S_{i,N} \cdot x_N = y_i$, hence $S_{i,N} = x_N^{-1} \cdot (y_i - S_{i,1} \cdot x_1 - \cdots - S_{i,N-1} \cdot x_{N-1})$

3.1.4 Security game for the attack on F^{WEAK}

Security game on the CPRF F^{WEAK}

The attacker $\mathcal{A}$ interacts with an oracle O which responds either with a $CPRF$ or with a random function. The goal of the attacker is to distinguish between the two cases.

Phase 1 - Initialization :

— O generates a master key $S \in (\mathbb{Z}/p\mathbb{Z})^{M \times N}$.

Phase 2 - Constrained key request :

— $\mathcal{A}$ requests a constrained key by sending $\vec{y} = (0, \ldots, 0, 1)$ in order to receive $S_{\vec{y}}$.

— O returns $S_{\vec{y}} = S + d \cdot \vec{y}^T$.

Phase 3 - Partial reconstruction :

— $\mathcal{A}$ computes $S_{\vec{y}} \cdot \vec{x}$ for each $\vec{x}$ corresponding to each row of X, in order to recover k_i, which allows them to recover $S_{i,j}, \forall j \leq N-1$.

Phase 4 - Dictionary initialization :

— $\mathcal{A}$ initializes an empty dictionary D.

Phase 5 - Evaluation queries and attack :

— $\mathcal{A}$ can then request the evaluation of several $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$ that are not orthogonal to $\vec{y}$ as follows :

  — Choose a random $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$ while ensuring that $x_N \neq 0$ in order to guarantee that $\vec{x}$ is not orthogonal to $\vec{y}$.

  — Request from O the evaluation of $\vec{x}$: $Eval(\vec{x})$.

  — Compute $S_{i,N} \forall i$; $\mathcal{A}$ can compute it because they know the values of $S_{i,1}, \ldots, S_{i,N-1}$.

  — If $(S_{1,N}, \ldots, S_{N,N})$ is not in D, then insert into D : $((S_{1,N}, \ldots, S_{N,N}), 0)$, otherwise increment $D(S_{1,N}, \ldots, S_{N,N})$

  — If $D(S_{1,N}, \ldots, S_{N,N}) = \max\{D(\cdot)\}$, then output CPRF oracle, otherwise output random function.

Phase 6 - Interpretation :
In this security game, for each request evaluating a vector $\vec{x}$, the attacker recomputes the values of $S_{1,N}, \ldots, S_{N,N}$. Repeated values have a high probability of being the true values of $S_{1,N}, \ldots, S_{N,N}$, which makes it possible to deduce that the evaluation result comes from a CPRF. If, after an evaluation, the recovered values of $S_{1,N}, \ldots, S_{N,N}$ are not repeated, then the result most likely comes from a random function.

This allows an attacker $\mathcal{A}$ to distinguish a CPRF from a random function, which shows that F^{WEAK} is not secure.

3.1.5 Strengthening $F^{\mathbf{WEAK}}$ using hashing

In order to improve the security of the CPRF F^{WEAK}, it is possible to implement it using a hash function H. This implementation modifies the evaluation of constrained and unconstrained inputs, without preventing an attacker from requesting a constrained key.

Evaluation on unconstrained inputs : When an attacker requests the evaluation of a vector $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$, the oracle returns $Eval_H(\vec{x}) = H(S \cdot \vec{x})$ instead of $Eval(\vec{x}) = S \cdot \vec{x}$, where H is a hash function.

Constrained key request : The constrained key request made by an attacker remains unchanged. The attacker sends a vector $\vec{y} \in (\mathbb{Z}/p\mathbb{Z})^N$, then the oracle returns $Constrain(S, \vec{y}) = S_{\vec{y}} = S + d \cdot \vec{y}^T$.

Evaluation on constrained inputs : After receiving a constrained key S_y, in order to evaluate a vector $\vec{x}$ orthogonal to $\vec{y}$, the attacker computes $EvalC_H(\vec{x}) = H(S_y \cdot \vec{x})$ instead of $EvalC(\vec{x}) = S_y \cdot \vec{x}$, where H is a hash function.

This implementation using a hash function H ensures that an attacker can still perform evaluations on constrained inputs, but they will no longer be able to reconstruct the matrix S by solving linear systems. Indeed, using the attack described in the previous section, recovering S amounts to finding a preimage of $H(S \cdot \vec{x})$. This confirms that applying a hash function ensures the security of the CPRF F^{WEAK}.

3.1.6 Security proof of hashed F^{WEAK}

Theorem 2 (CJ25 [2])

Let CF be a CPRF. If the following conditions are satisfied :

— $\min_x |\mathsf{CF.Out}(\lambda, x)|$ is super-polynomial in λ,

— CF is IND*-NC-CPRF secure,

— The random oracle used is non-programmable,

then the construction $\mathsf{Hashed}[CF]$ is **SIM*-AC-CPRF secure**, meaning that no efficient adversary (with a polynomial number of queries) can distinguish this construction from an ideal one, even in the presence of a random hash oracle.

Super-polynomial function

A function $f : \mathbb{N} \to \mathbb{N}$ is said to be *super-polynomial* if, for every polynomial $p(\lambda)$, there exists N such that for all $\lambda \geq N$,

$$f(\lambda) > p(\lambda).$$

In other words, $f(\lambda)$ grows faster than any polynomial function.

— $\min_x |\mathsf{F}^{\mathbf{WEAK}}\mathbf{.Out}(\lambda, x)|$ **is super-polynomial in** λ **:**

In the implementation of the CPRF F^{WEAK}, we work with coefficients in $\mathbb{Z}/p\mathbb{Z}$ where p has size λ. Therefore, $|\mathbb{Z}/p\mathbb{Z}| \approx 2^\lambda$.

The evaluation of an input $x \in (\mathbb{Z}/p\mathbb{Z})^N$ by F^{WEAK} is performed as $Eval(F^{WEAK}, \vec{x}) = S \cdot \vec{x}$, which is a linear map and therefore a bijection. This implies that $\mathsf{F}^{\mathrm{WEAK}}.\mathsf{Out}(\lambda, x)$ has the same size for all $x \in (\mathbb{Z}/p\mathbb{Z})^N$.

Fix $x \in (\mathbb{Z}/p\mathbb{Z})^N$. For all $S, S' \in (\mathbb{Z}/p\mathbb{Z})^{M\times N}$, $S \neq S' \Rightarrow S \cdot x \neq S' \cdot x$. Since S has size $M \times N$, and the coefficients belong to $\mathbb{Z}/p\mathbb{Z}$, we deduce that there exist $(2^\lambda)^{M\times N} = 2^{\lambda\times M\times N}$ distinct master keys.

We conclude that $\min_x |\mathsf{F}^{\mathrm{WEAK}}.\mathsf{Out}(\lambda, x)| \approx 2^{\lambda\times M\times N}$, which is super-polynomial in λ.

— **The random oracle used is non-programmable :**

Recall that the lazy sampling function $P.Ls$ and the programming function $P.Prog$ are implemented as follows :

$\underline{P.Ls(1^\lambda, x, \sigma_P)}$	$\underline{P.Prog(1^\lambda, x, y, \sigma_P)}$
IF $\sigma_P[x] = \perp$ then $\sigma_P[x] \leftarrow Random(1^\lambda)$	If $\sigma_P[x] = \perp$ then $\sigma_P[x] \leftarrow y$
Return $\sigma_P[x]$	Return $\perp$

We say that P is non-programmable if $P.Prog$ does not update σ_P and the execution of $P.Ls$ on different inputs does not change the distribution of its output.

In our implementation of F^{WEAK}, we use lazy sampling to implement the hash function H. This implies that $Eval_H(\vec{x}) = H(S \cdot \vec{x})$ is fixed only when the evaluation of $\vec{x}$ is requested, but not earlier.

We can therefore conclude that P is non-programmable.

— $F^{\mathbf{WEAK}}$ **is IND*-NC-CPRF secure :**

For this condition, we can prove that CF is IND*-NC-CPRF secure if there does not exist an attacker who :

— Requests all evaluations
— Performs their computations
— Returns their answers to the oracle while having a significant advantage

In our security game, during phase 5, the attacker proceeds as follows :

— Requests **one** evaluation
— Performs their computations
— Returns their answer to the oracle while having a significant advantage
— Repeats

However, it is possible to modify phase 5 of our security game so that the attacker proceeds as follows :

- Choose n random vectors $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$ while ensuring that $x_N \neq 0$ for each vector $\vec{x}$ in order to guarantee that $\vec{x}$ is not orthogonal to $\vec{y}$.
- Request from O the evaluation of the n vectors $\vec{x}$: $Eval(\vec{x})$.
- For each evaluation of the n vectors $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$, compute $S_{i,N} \forall i$.
- For each $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$, if $(S_{1,N}, \ldots, S_{N,N})$ is not in D, then insert into D : $((S_{1,N}, \ldots, S_{N,N}), 0)$, otherwise increment $D(S_{1,N}, \ldots, S_{N,N})$
- For each $\vec{x} \in (\mathbb{Z}/p\mathbb{Z})^N$, if $D(S_{1,N}, \ldots, S_{N,N}) = \max\{D(\cdot)\}$, then output CPRF oracle, otherwise output random function.

An attacker will have the same advantage using this attack without performing adaptive evaluations, which shows that F^{WEAK} is not IND*-NC-CPRF secure.

However, the fact that F^{WEAK} is not IND*-NC-CPRF secure does not imply that $\mathsf{Hashed}[F^{\text{WEAK}}]$ is not **SIM*-AC-CPRF secure**. Therefore, this is not a proof that $\mathsf{Hashed}[F^{\text{WEAK}}]$ is not **SIM*-AC-CPRF secure**.

3.2 Rabin trapdoor permutation

The Rabin permutation is a trapdoor function based on the hardness of integer factorization. It relies on the squaring operation modulo a composite integer.

- **Key generation :**

  We choose two distinct prime numbers p and q such that :

  $$p \equiv q \equiv 3 \pmod 4$$

  The public key is defined as :

  $$n = p \cdot q$$

  The private key is the pair (p, q).

  This condition on p and q allows efficient computation of square roots modulo these prime numbers.

- **Rabin permutation :**

  The Rabin permutation is the function :

  $$f : \mathbb{Z}_n \to \mathbb{Z}_n$$

  defined by :

  $$f(x) = x^2 \bmod n$$

  This function is easy to compute, but difficult to invert without knowing the factorization of n.

- **Inverting the Rabin permutation :**

  Inverting the Rabin permutation amounts to solving the equation :

  $$x^2 \equiv y \pmod n$$

  When $n = p \cdot q$, this equation admits exactly **four distinct solutions** modulo n.

  We begin by reducing the problem modulo p and q :

  $$\begin{cases} x^2 \equiv y \pmod p \\ x^2 \equiv y \pmod q \end{cases}$$

  Since $p \equiv 3 \pmod 4$, a square root modulo p is given by :

  $$r_p = y^{\frac{p+1}{4}} \bmod p$$

  Similarly, modulo q :

  $$r_q = y^{\frac{q+1}{4}} \bmod q$$

The solutions are therefore :

— $X_0 = (r_p * q * q^{-1}_{\bmod p} + r_q * p * p^{-1}_{\bmod q}) \bmod n$
— $X_1 = (r_p * q * q^{-1}_{\bmod p} - r_q * p * p^{-1}_{\bmod q}) \bmod n$
— $X_2 = (-r_p * q * q^{-1}_{\bmod p} + r_q * p * p^{-1}_{\bmod q}) \bmod n$
— $X_3 = (-r_p * q * q^{-1}_{\bmod p} - r_q * p * p^{-1}_{\bmod q}) \bmod n$

Problem : Which of these roots corresponds to the original message ?

The function $f(x) = x^2 \bmod n$ is not injective : each ciphertext c has 4 preimages. Even with the secret key (p, q), it is possible to compute all the roots but **not directly determine which one corresponds to the original message**. Applying the inverse $n - c$ times does not guarantee recovering the original message.

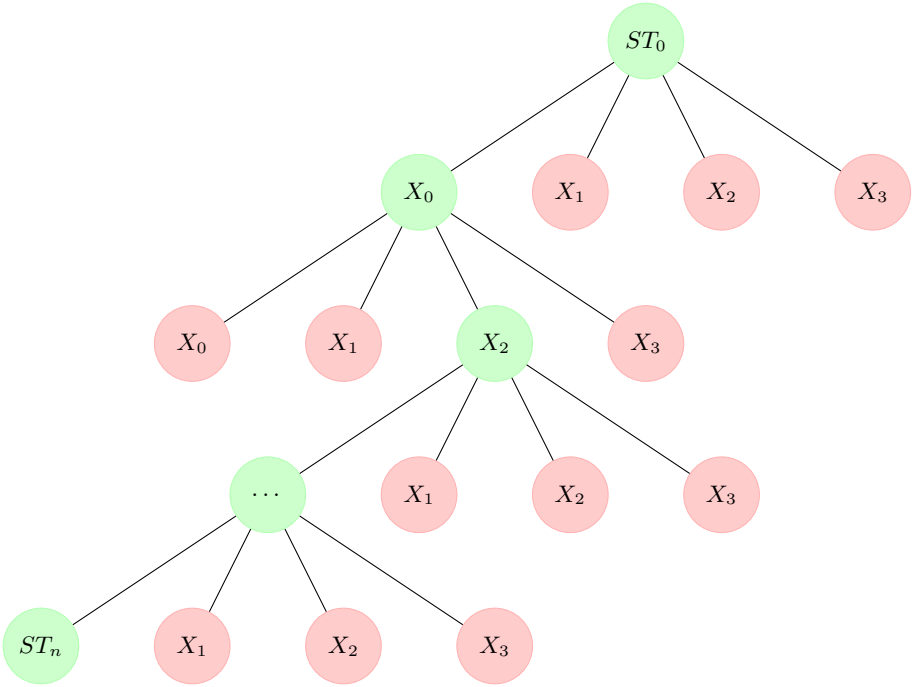


FIGURE 5 – Tree of preimages in the Rabin permutation.

Conclusion : The Rabin permutation is not suitable for our CPRF because each value has four possible preimages. Even with the private key, it is impossible to know which one corresponds to the original input, which prevents the repeated application of the inverse $(n - c)$ times from correctly recovering the CPRF output.

3.3 AES trapdoor permutation

AES encryption applies a bijective function $E_k(x)$ that depends on the key k. Unlike $x \mapsto x^2 \bmod n$, each encrypted block has a **unique preimage** for a given key. However, in order to decrypt, **the key k must absolutely be known** :

— Without the key, the attacker cannot invert the permutation.
— AES remains secure thanks to the secret key.

Therefore, the use of AES as a trapdoor permutation is not suitable for our CPRF.

4 The CJ25 attack

4.1 General description and intuition behind the attack

The CJ25 attack exploits the fact that an attacker can obtain a constrained key allowing them to evaluate the pseudorandom function on a subset of inputs, while still retaining access to a global evaluation oracle.

First, the adversary chooses an index n and requests from the oracle a constrained key (PK, ST_n, n). This key allows them to evaluate the function on any input x such that $x < n$ using the associated public permutations.

In a second step, the adversary queries the evaluation oracle on an input $x < n$ and obtains a value ST_x. At this stage, the attacker does not know whether ST_x comes from a constrained pseudo-random function or from a random function.

Using the constrained key, the attacker successively applies the public permutations in order to compute the value $\pi_{PK}^{x-n}(ST_x)$. If the attacker recovers $ST_a = ST_x$, they can then conclude that the value ST_x is the result of a constrained pseudo-random function.

It is therefore possible for an attacker to obtain information about the structure of the CPRF from the constrained key and the obtained evaluations, which allows them to distinguish a CPRF from a random function with non-negligible probability.

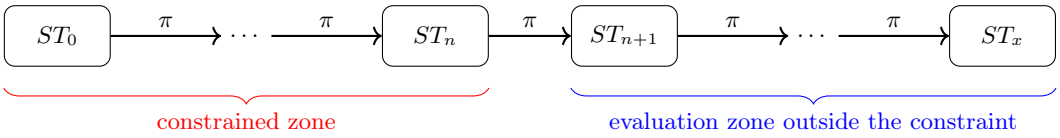


FIGURE 6 – General diagram of the CJ25 attack

4.2 Security game model and attacker advantage

Security game for CPRFs

The attacker $\mathcal{A}$ interacts with an oracle O which is either a $CPRF$ or a random function. The goal of the attacker is to distinguish between the two cases.

- **Phase 1 :** $\mathcal{A}$ requests a constrained key $K_C = (PK, ST_n, n)$ for a constraint n of their choice, allowing them to evaluate the function on a subset of inputs.
- **Phase 2 :** $\mathcal{A}$ makes several evaluation queries to the oracle O on inputs $x > n$ in order to obtain unconstrained values ST_x.
- **Phase 3 :** $\mathcal{A}$ uses the obtained values to evaluate $ST_a = \pi_{PK}^{n-x}(ST_x)$ and compares it with ST_n.
- **Decision :** $\mathcal{A}$ decides that O is the $CPRF$ if $ST_a = ST_n$, otherwise they decide that O is a random function.

In the case where O is a $CPRF$, the attacker $\mathcal{A}$ always recovers $ST_a = ST_n$, and never makes a mistake. On the other hand, if O is a random function, the probability that $ST_a = ST_n$ is $1/N$, where N is the size of the output space of the function. In this case, the attacker makes a mistake with probability $1/N$. The attacker $\mathcal{A}$ therefore succeeds with probability $1 - 1/N$ in correctly distinguishing the two cases, which is non-negligible for reasonable values of N.

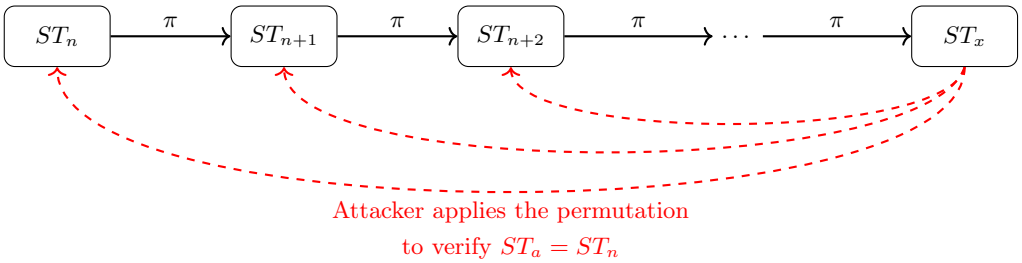


FIGURE 7 – Diagram of the permutations used in the attack within the unconstrained zone

4.3 Experimental implementation of the attack

To simulate the security game, we implemented a local client–server architecture based on TCP. The server implements the game oracle (evaluation and constrained key generation), while the client plays the role of the attacker (evaluation and constraint requests, tests, ...).

4.3.1 Implementation of the security oracle

The oracle provides two interfaces accessible to the attacker :

— `EVAL(x)` : returns a value associated with the input x.
In the PRF world, the oracle returns the evaluation of the CPRF using the master key.
In the random world, it returns a uniformly random value of the same size.

— `CONSTRAIN(n)` : returns a constrained key associated with index n, allowing the evaluation of the function on a subset of inputs.

4.3.2 Implementation of the attacker and strategy

The attacker chooses an index n and requests from the oracle the corresponding constrained key. This key allows them to evaluate the function on inputs strictly greater than n.

They then perform `EVAL` queries on inputs $x > n$. Using the values returned by the oracle and the constrained key, they apply the public permutations in order to test the consistency of the CPRF structure.

If the attacker finds a match, they can conclude that the oracle implements a CPRF rather than a random function.

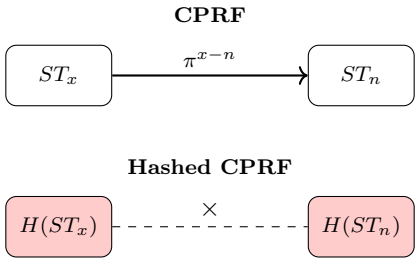
5 Strengthening the CPRF using hashing

In order to improve the security of the BMO17 construction against the CJ25 attack, it is possible to implement a hashed version of the CPRF.

5.1 General principle of the hashed CPRF

The idea is to introduce a cryptographic hash function into the evaluation process. The attacker will still be able to evaluate the CPRF on unconstrained inputs, but they will no longer be able to exploit the results to distinguish the CPRF from a random function because these results will be irreversibly hashed.

→ The role of the hash function is to make any use or comparison through the public permutations impossible.



Implementation :

We implemented two versions of the hashed CPRF :

— **Simple hashed CPRF** : the evaluation is performed normally, then the result is hashed using `SHA-256` before being returned.
  — $Eval_H(x)$: returns the hash of the evaluation on input x using the master key, that is $H(Eval((SK, ST_0), x))$.
  — $EvalC_H(n)$: returns the hash of the evaluation on input n using the constrained key, that is $H(EvalC((PK, ST_n, n), c))$.

— **Hashed CPRF with lazy sampling** : in order to get closer to the theoretical model of an ideal hash oracle used in the CJ25 paper, we also implemented a version based on the lazy sampling technique described in the following section.

5.2 Explicit construction of the Cheng and Jaeger hashed CPRF

We present here the hashed version proposed in the CJ25 paper.

Notation used :

- P : ideal hash function modeled as a random hash oracle.
- σ_P : internal state of P (initially empty) storing the pairs (x, y).
- λ : security parameter (output size of P).
- k : secret key of the CPRF.
- k_R : constrained key associated with a set of restrictions R.
- x : function input.
- $y \in \{0,1\}^\lambda$: hash value associated with an input.

The evaluation is defined by :

$$\mathsf{Hashed[CPRF].Eval}(1^\lambda, k, x) \;=\; P.\mathsf{LazySampling}(1^\lambda, \mathsf{CPRF.Eval}(1^\lambda, k, x) : \sigma_P).$$
$$\mathsf{Hashed[CPRF].EvalC}(1^\lambda, k_R, R, x) \;=\; P.\mathsf{LazySampling}(1^\lambda, \mathsf{CPRF.EvalC}(1^\lambda, k_R, R, x) : \sigma_P).$$

Lazy Sampling

Search for x in σ_P
If $x \notin \sigma_P$:
  Choose $y \leftarrow \{0,1\}^\lambda$ uniformly at random
  Store (x, y)
Otherwise :
  Retrieve the value y associated with x
Return y

Lazy sampling algorithm for an ideal primitive.

6 Experimental Results

After implementing different variants of the CPRF (original version, hashed version with SHA-256, and hashed version with lazy sampling), we conducted experiments in order to evaluate the effectiveness of the CJ25 attack on each of them. The tests consist of executing 100 evaluation queries for each version and observing the evolution of the attack success rate throughout the queries.

Figure 8 presents the results obtained on the non-hashed version of the CPRF. We observe that the CJ25 attack is highly effective in this case, with a success probability close to 1 from the very first queries. This is explained by the fact that the structure of the CPRF remains fully exploitable in this model, allowing the attacker to distinguish the function's behavior.

In Figures 9 and 10, the same attack is applied to the hashed versions (SHA-256 and lazy sampling). We then observe a systematic failure of the attack, with a success rate close to 0.5, corresponding to behavior equivalent to random guessing. In these cases, the attacker is no longer able to exploit the structure of the CPRF.

Finally, the results suggest that the choice of hashing scheme (SHA-256 or lazy sampling) has no significant impact on the robustness observed in our experiments, as both variants exhibit similar behavior against the CJ25 attack.

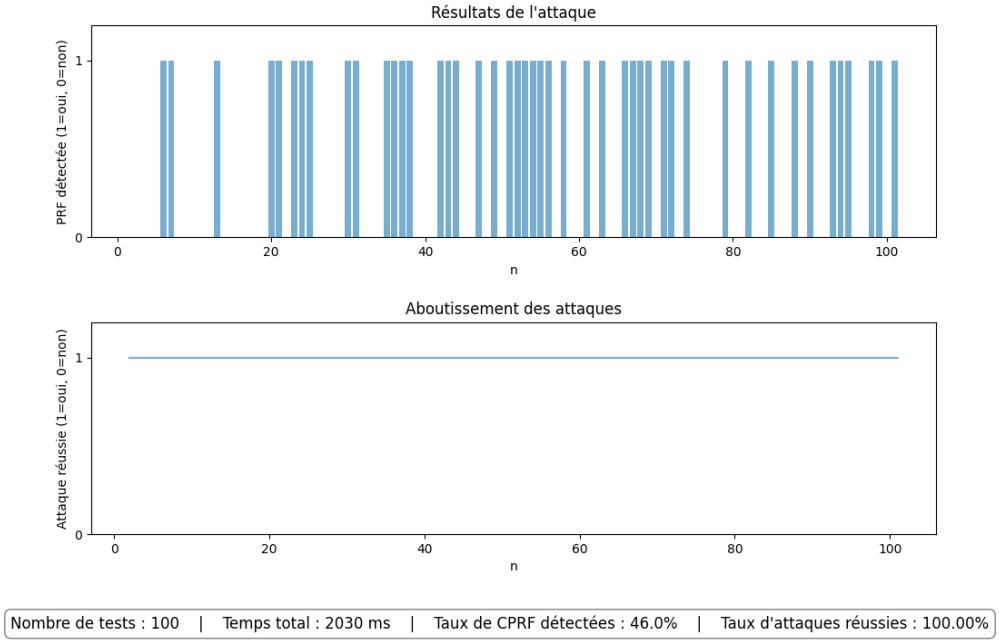


Figure 8 – Attack on the non-hashed version

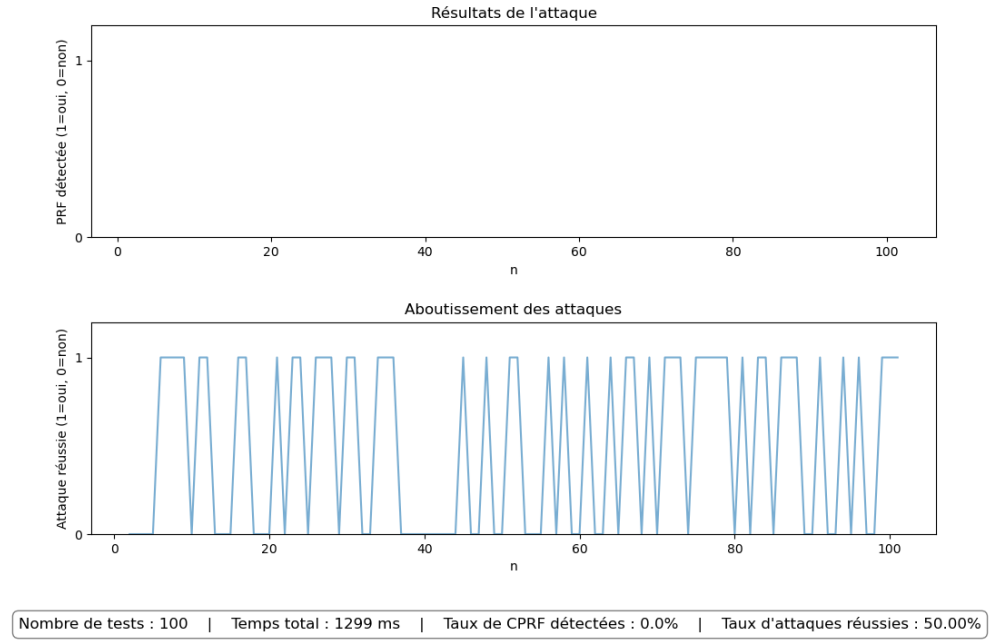


Figure 9 – Attack on the hashed version with SHA-256

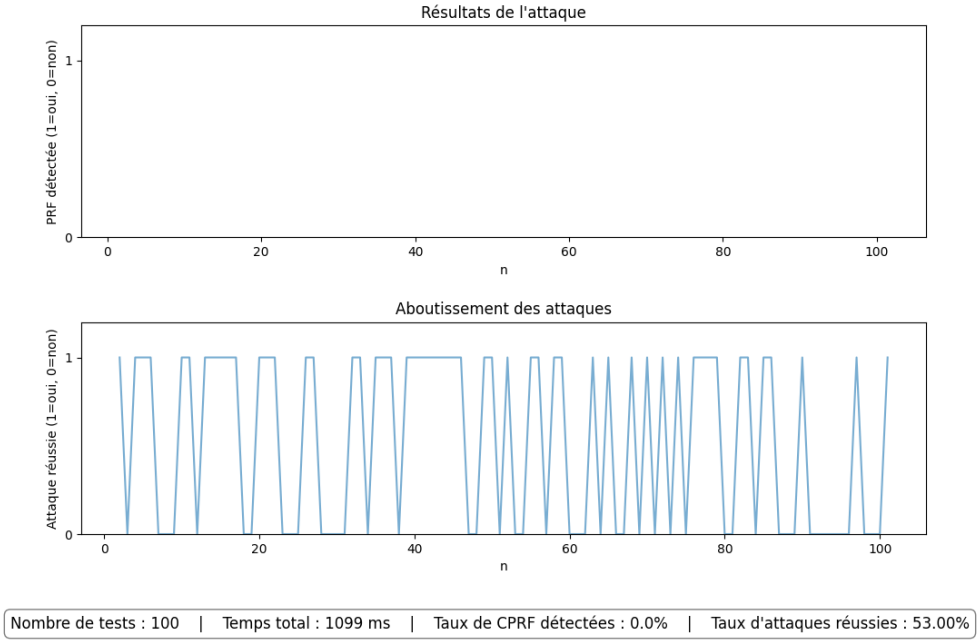


FIGURE 10 – Attack on the hashed version with lazy sampling

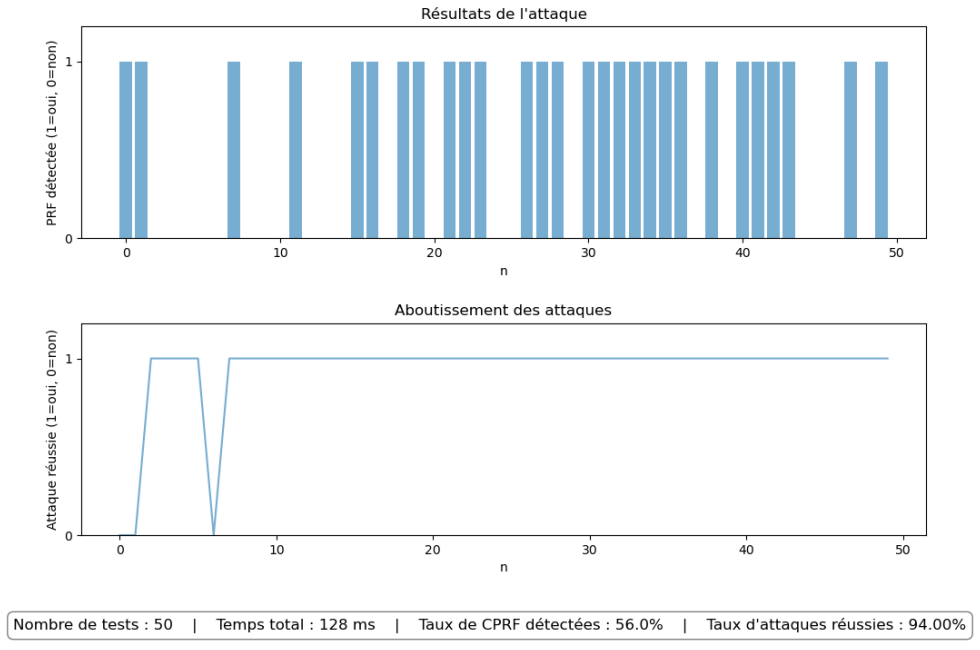


FIGURE 11 – Attack on FWEAK

Références

[1] E. Bost, B. Minaud, and O. Ohrimenko, *Forward and Backward Private Searchable Encryption from Constrained Cryptographic Primitives*, ACM CCS 2017.

[2] K. Cheng and J. Jaeger, *Adaptive Security for Constrained PRFs*, 2025.